

SIGma Fall 2026

Welcome (Back)!

Ian Chen, Mihir Tandon, Franklin Zhang



Outline

Admins (in No Particular Order)

Computational Geometry

CG:SHOP

Meetings



Housekeeping



- Join the Discord!



Section 1

Admins (in No Particular Order)



Ian

- Senior in Stats & CS
- CS 473 course assistant
- General interests: graph algorithms, parallel computing, algorithmic engineering



Mihir

- Junior in ECE + Math
- CS 374 course assistant
- General interests: compilers, HPC, algorithms, hardware



Franklin

- old grad student



Section 2

Computational Geometry



Applications

- Robotics; TAMP
- Computer graphics
- Geographic information systems (GIS)
- Computer-aided {design, manufacturing} (CAD/CAM)



Convex hulls.



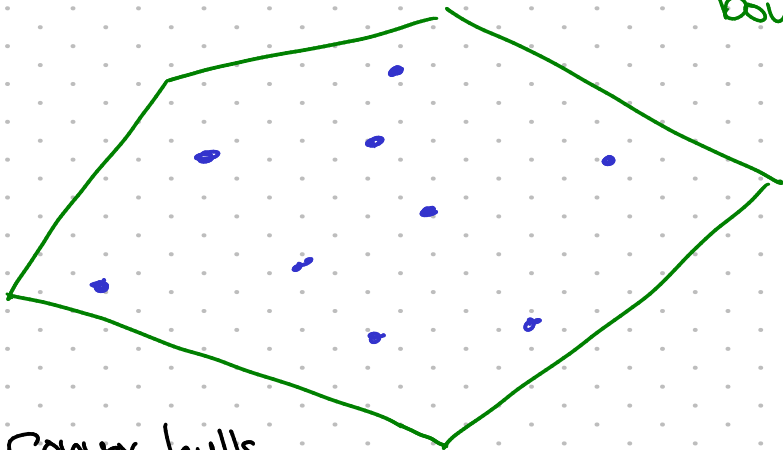
points in 2D space



Convex hulls.



points in 2D space
bounding polygon

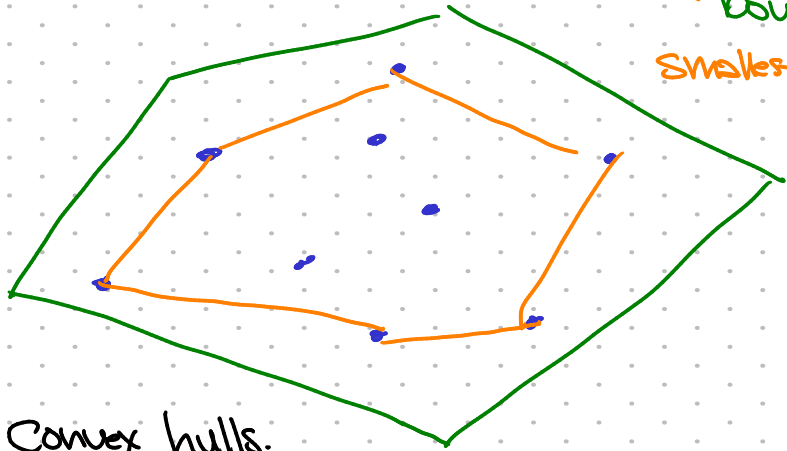


Convex hulls.



points in 2D space

✓ bounding polygon
Smallest "convex"



Convex hulls.



Convex hulls

- Input: n points in 2D space
- Output smallest convex bounding polygon

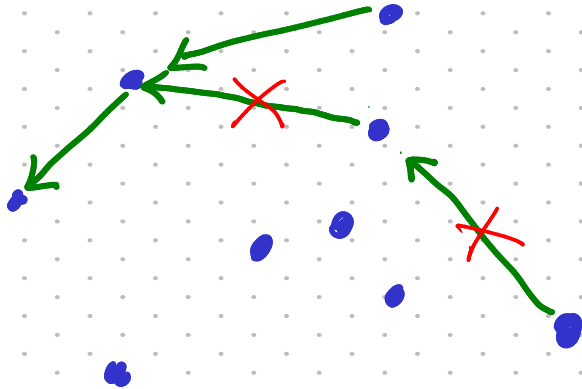


Convex hulls

- Input: n points in 2D space
 - ▶ an unordered list of (x, y) pairs
- Output smallest convex bounding polygon
 - ▶ an ordered list of (x, y) pairs
 - ▶ subset of the input; in CCW order

```
Point = namedtuple("Point", ["x", "y"])  
def upper_hull(points: set[Point]) -> list[Point]:  
    ...
```



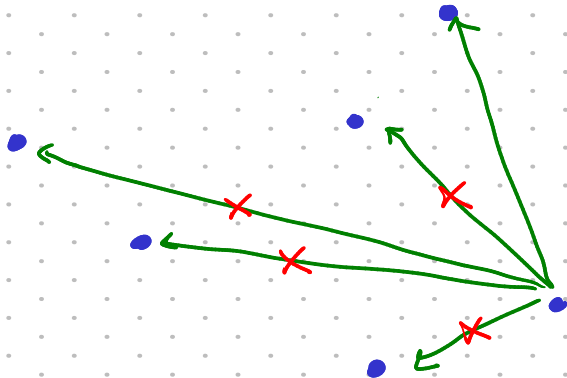


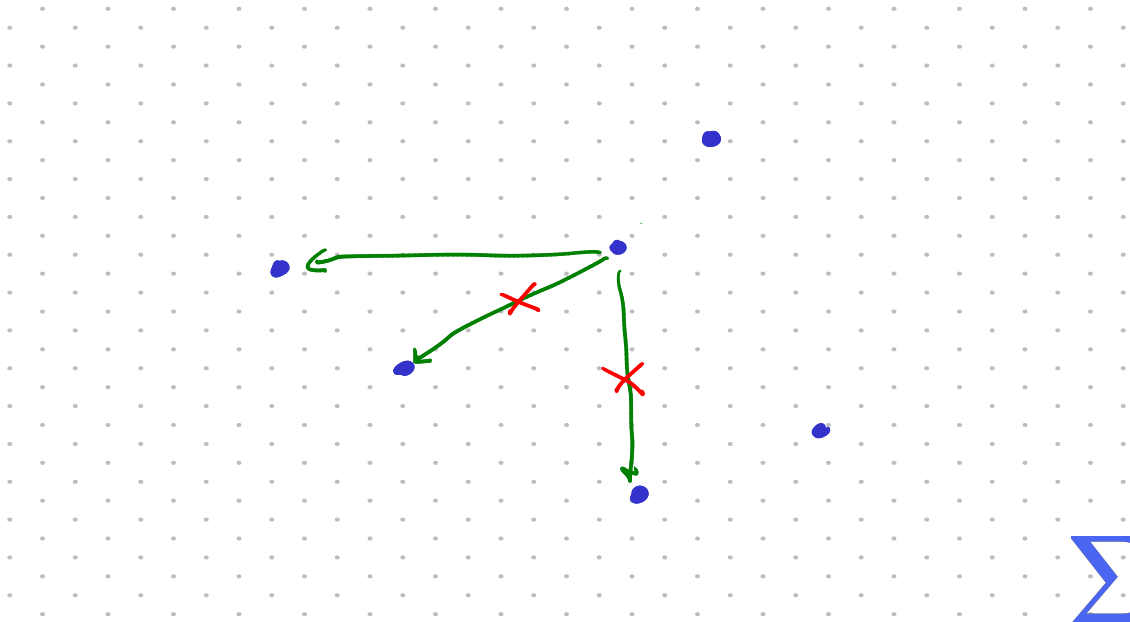
Σ

Brute Force

```
def upper_hull_1(points: set[Point]) -> list[Point]:  
    lines = []  
    for p1, p2 in combinations(points, 2):  
        p1, p2 = max(p1, p2), min(p1, p2)  
        if all(point_below_line((p1, p2), q)  
                for q in points):  
            lines.append((p1, p2))  
    return sorted(set(flatten(lines)), reverse=True)
```



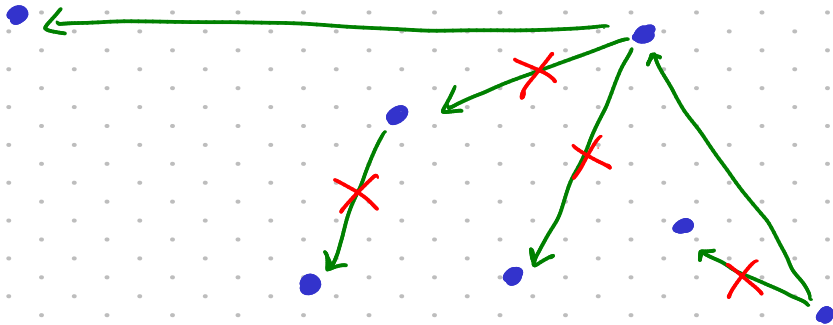




A little smarter...

```
def upper_hull_2(points: set[Point]) -> list[Point]:
    lines = []
    for p1 in points - {min(points)}:
        p2 = max((q for q in points if q < p1),
                  key=lambda q: sin_angle(p1, q))
        if all(point_below_line((p1, p2), q)
                for q in points):
            lines.append((p1, p2))
    return sorted(set(flatten(lines)), reverse=True)
```





Sweeping

```
def upper_hull_3(points: set[Point]) -> list[Point]:
    hull = []
    for point in sorted(points, reverse=True):
        while len(hull) >= 2 and
            not point_below_line(
                (hull[-2], hull[-1]), point):
            hull.pop()
        hull.append(point)
    return hull
```



Section 3

CG:SHOP

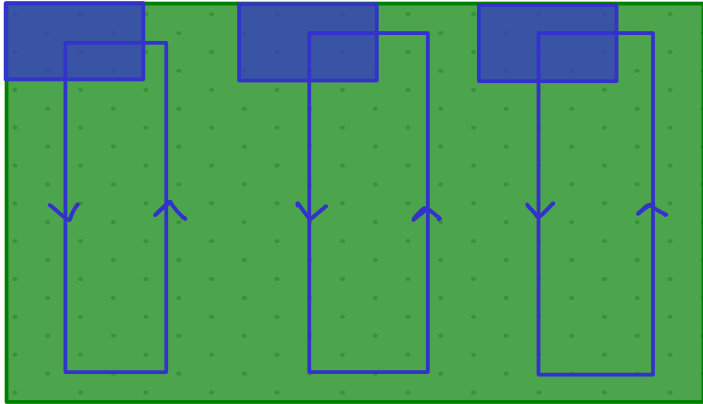


What is CG:SHOP?

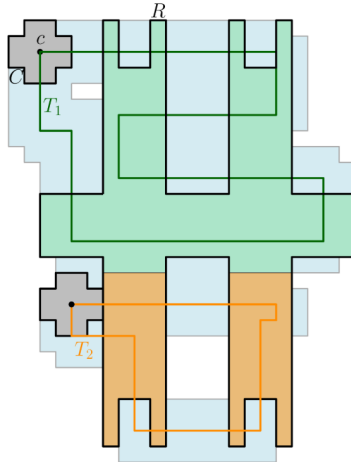
- **CG:SHOP** (Computational Geometry: Solving Hard Optimization Problems) is a computation geometry competition for students and researchers.
- Each year, teams build solvers for a difficult geometric optimization problem.
- The 2027 challenge is **Multi-Robot Lawn Mowing for Polyominoes**.



Multi-robot lawn mowing.



An Example



An example polyomino region (light green & light orange), a small polyomino cutter with center (gray), a (suboptimal) pair of valid tours (dark green and orange) and the region it covers (light blue).



Timeline

- Example instances and Python utilities are already available.
- The competition starts on October 15, 2026.
- Final submissions are due January 31, 2027.
- Discord for CG:SHOP problems: discord.gg/3FXC6xzzWe.
- We might also have a professor advise us, this is still TBD.



CG:SHOP Logistics

- [challenge website](#).
- (Soft) Pre-requisites: data-structures and algorithms at the CS 225 level.
- Roughly split into *algorithm* and *implementation* teams.
 - ▶ *Algorithm* teams focus on literature review, understanding proofs, and proposing algorithms.
 - ▶ *Implementation* teams focus on implementing algorithms, and proposing algorithms.
 - ▶ Both teams need a strong understanding of current research in this area.



Section 4

Meetings



Standard Meetings

- ≈ 30 minutes for a *lecture*, plus questions
 - ▶ Present a problem, or approaches, in computational geometry
 - ▶ Spreadsheet for presentation signups: TODO
 - ▶ Admins are very willing to help presenters.
 - ▶ Resources: jeffe.cs.illinois.edu/teaching/compgeom/resources.html
- ≈ 10 minutes for project updates and discussion



Project-Centered Meetings

- $\approx$ 10 minutes for a *standup*, where each team shares recent progress.
- $\approx$ 30 – 40 minutes for focused work time.
- Don't feel like you are siloed into one time, you can move between teams whenever you like.



Project AI Policy

- You must be able to present and justify every decision you make; communicate your ideas!
 - ▶ This is a learning project!
 - ▶ Have fun with learning about cool geometric algorithms.
 - ▶ We will not be merging any code that you can not explain.
- You are free to use AI for understanding papers, setting up environments, or generating algorithmic ideas, but treat its output with caution.



Goals for next meeting? (09/14)

